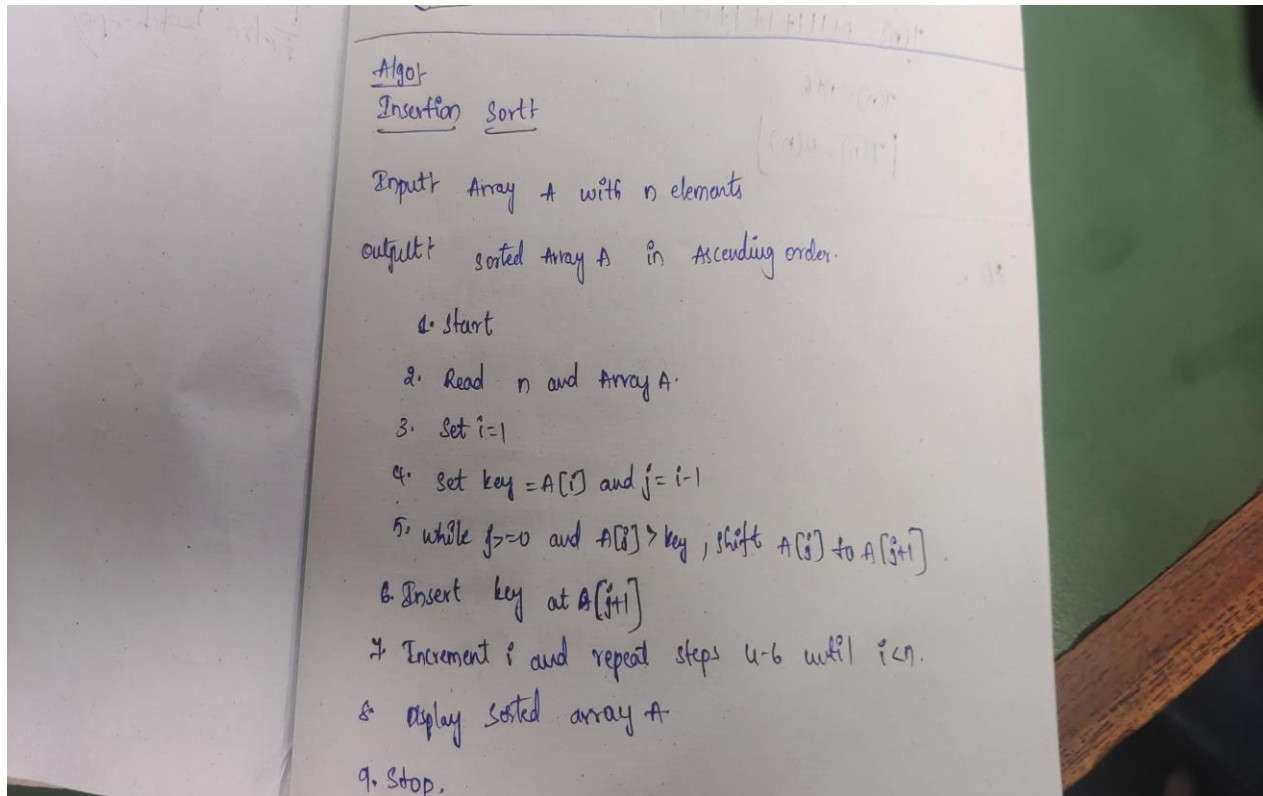


Aim: To Implementation and time analysis of Insertion sort algorithm.

Algorithm for Insertion sort:



Program of Insertion sort:

```
#include <iostream>
using namespace std;

class InsertionSortApp
{
public:
    int arr[100], n;

    void arrayInput()
```

```
{  
    cout << "Enter number of elements: ";  
    cin >> n;
```

```
  
    cout << "Enter the elements: ";  
    for (int i = 0; i < n; i++)  
    {  
        cin >> arr[i];  
    }  
}
```

```
void printArray()  
{  
    for (int i = 0; i < n; i++)  
    {  
        cout << arr[i] << " ";  
    }  
    cout << endl;  
}
```

```
// Insertion Sort Function  
void sort()  
{  
    for (int i = 1; i < n; i++)  
    {  
        int key = arr[i];  
        int j = i - 1;  
  
        while (j >= 0 && arr[j] > key)  
        {  
            arr[j + 1] = arr[j];
```

```
        j--;  
    }  
  
    arr[j + 1] = key;  
    }  
};  
  
int main()  
{  
    InsertionSortApp is;  
  
    is.arrayInput();  
  
    cout << "\nArray before sorting: ";  
    is.printArray();  
  
    is.sort();  
  
    cout << "\nArray after Insertion Sort: ";  
    is.printArray();  
  
    return 0;  
}
```

Output:

```
Output

Enter number of elements: 5
Enter the elements: 2
1
8
4
3

Array before sorting: 2 1 8 4 3

Array after Selection Sort: 1 2 3 4 8

=== Code Execution Successful ===
```

Time complexity:

Time Complexity

for () { $\rightarrow n$

 key = arr[i]; $\rightarrow 1$

 j = i-1; $\rightarrow 1$

 while () { $\rightarrow n$

 arr[j+1] = arr[j]; $\rightarrow 1$

 j-- $\rightarrow 1$

 } $\rightarrow 1$

 arr[i+1] = key; $\rightarrow 1$

}

$T(n) = (n+1) + (n+1+1) + \dots + (n+2) + (n+3) + \dots + (n^2 + 3n + 2n + \dots)$

$T(n) = O(n^2)$